

МИНОБРНАУКИ РОССИИ
Федеральное государственное бюджетное образовательное
учреждение высшего образования
«Ижевский государственный технический университет имени М.Т. Калашникова»
(ФГБОУ ВО «ИжГТУ имени М.Т. Калашникова»)

Факультет «Информационные технологии»
Кафедра «Автоматизированные системы обработки информации и управления»
Направление подготовки 09.03.01 «Информатика и вычислительная техника»

ОТЧЕТ ПО ПРАКТИКЕ

Учебная практика. Эксплуатационная
(наименование типа практики)

ИжГТУ имени М.Т. Калашникова
(полное наименование профильной организации)

Выполнил обучающийся _____ / Фертиков.Н.А, 2 курс, Б24-782-1 /
(подпись) (ФИО, курс, номер группы)

Дата сдачи отчета: «__»_____ 2026 г.

Дата аттестации «__»_____ 2026 г.

Оценка _____

Руководитель практики от
ИжГТУ имени М.Т. Калашникова _____ / Н.С. Исупов, ст. преподаватель каф. АСОИУ/
(подпись) (И.О. Фамилия, должность,
ученая степень)

Заведующий кафедрой _____ / М.Н. Мокроусов, к.т.н. /

Содержание

1	Задание на практическую работу.....	4
2	Разработка и описание метода поиска оптимального решения	4
3	Иерархия классов	6
4	Схемы алгоритмов.....	12
5	Результаты тестирования программы на различных наборах данных	16
	Список используемых источников	21
	Приложение А «Исходный текст на языке C#»	22

1 Задание на практическую работу

Имеются номера четырех типов: одноместный, двухместный, трехместный и VIP номер, отличающиеся стоимостью проживания за сутки. Количество номеров ограничено (н-р: одноместных 30, двухместных 20, трехместных 10 и VIP 5). Номера могут бронироваться на сутки, неделю, месяц. Часть номеров изначально занята (количество занятых номеров, и на какой срок - придумать самостоятельно). Забронировавшим клиентам необходимо предоставить номер не позднее 3х суток с момента бронирования. В конце каждого дня система должна распределить максимальное количество поступивших заказов с учетом доступных номеров (поступивших за сутки заказов всегда чуть больше, чем доступных по заданному типу номеров, т.е. н-р, доступно 6 одноместных номеров, поступило 8 заказов на одноместные номера).

2 Разработка и описание метода поиска оптимального решения

Целью проекта является автоматизация процесса распределения заявок на бронирование номеров гостиницы с ограниченным номерным фондом и избыточным потоком клиентов. Основной критерий оптимизации – максимизация количества обслуженных заявок. При этом система должна учитывать различные типы номеров (одноместный, двухместный, трёхместный, VIP), возможность предоставления номеров меньшей вместимости при отсутствии требуемых (даунгрейд), а также допустимое время ожидания заселения — не более трёх суток с момента подачи заявки.

Метод поиска решения основан на приоритетном жадном алгоритме распределения номеров с возможностью даунгрейда и апгрейда. Выбор жадной стратегии обоснован необходимостью быстрого принятия решений в условиях динамического поступления заявок и ограниченных вычислительных ресурсов (симуляция «день за днём»).

Основные положения метода включают следующее:

- 1) формирование очереди заявок. В течение дня генерируются новые заявки случайным образом, причём количество заявок по каждому типу номеров превосходит количество свободных мест, что моделирует избыточный спрос. Заявки, не обслуженные в текущем дне, переносятся в общую очередь и могут быть удовлетворены в последующие дни в пределах трёхсуток. Просроченные заявки помечаются как отклонённые и исключаются из дальнейшего распределения», что точнее отражает поведение;

2) приоритизация заявок. Перед распределением все необработанные заявки сортируются по двум критериям в порядке значимости, по возрастанию проживания (в первую очередь обрабатываются 1 дневные заявки, потом 7 дневные, затем 30 дневные), при равной длительности – по времени поступления заказа (кто раньше заказал, тот в приоритете);

3) жадное распределение номеров. Для каждой заявки из отсортированного списка выполняется попытка подобрать размещение, исходя из текущего множества свободных номеров. Выбор осуществляется локально оптимально: назначается комбинация номеров, точно удовлетворяющая запрошенную вместимость, без пересмотра ранее сделанных назначений. Этот подход характерен для жадных алгоритмов — на каждом шаге фиксируется решение, которое кажется наилучшим в данный момент, без возвратов и глобального перебора всех возможных комбинаций заявок.

Правила подбора номеров и осуществления даунгрейда и апгрейда установлены следующим образом:

1) заявки на одноместные номера могут быть обслужены одним одноместным номером или одним двухместным;

2) заявки на двухместные номера в первую очередь обслуживаются одним двухместным номером, при его отсутствии проверяется наличие одного свободного трехместного или двух одноместных номеров;

3) заявки на трёхместные номера обслуживаются в первую очередь, одним трёхместным номером, при его отсутствии свободным VIP номером или двухместным и одноместным номером, или тремя одноместными номерами;

4) VIP-заявки обслуживаются только VIP-номерами, даунгрейд на обычные номера не производится. Только при успешном нахождении одной из комбинаций номеров заявка удовлетворяется, в противном случае она остаётся в очереди.

Такой иерархический подбор гарантирует, что номера большей вместимости резервируются для заявок, которые не могут быть удовлетворены меньшими номерами, и при этом минимизируется избыточность предоставляемых мест.

Разработанный алгоритм позволяет в режиме реального времени принимать решения о заселении, приближаясь к оптимальному по критерию минимизации простоя. Жадная природа алгоритма делает его быстрым и масштабируемым для большего числа номеров,

сохраняя при этом разумное качество распределения благодаря тщательно подобранным правилам приоритета, даунгрейда и апгрейда.

3 Иерархия классов

Программная система построена на основе объектно-ориентированного подхода и разделена на несколько логических слоёв представленных ниже на рисунке 1. Такое разделение обеспечивает соблюдение принципа единственной ответственности, упрощает модификацию и тестирование отдельных компонентов.

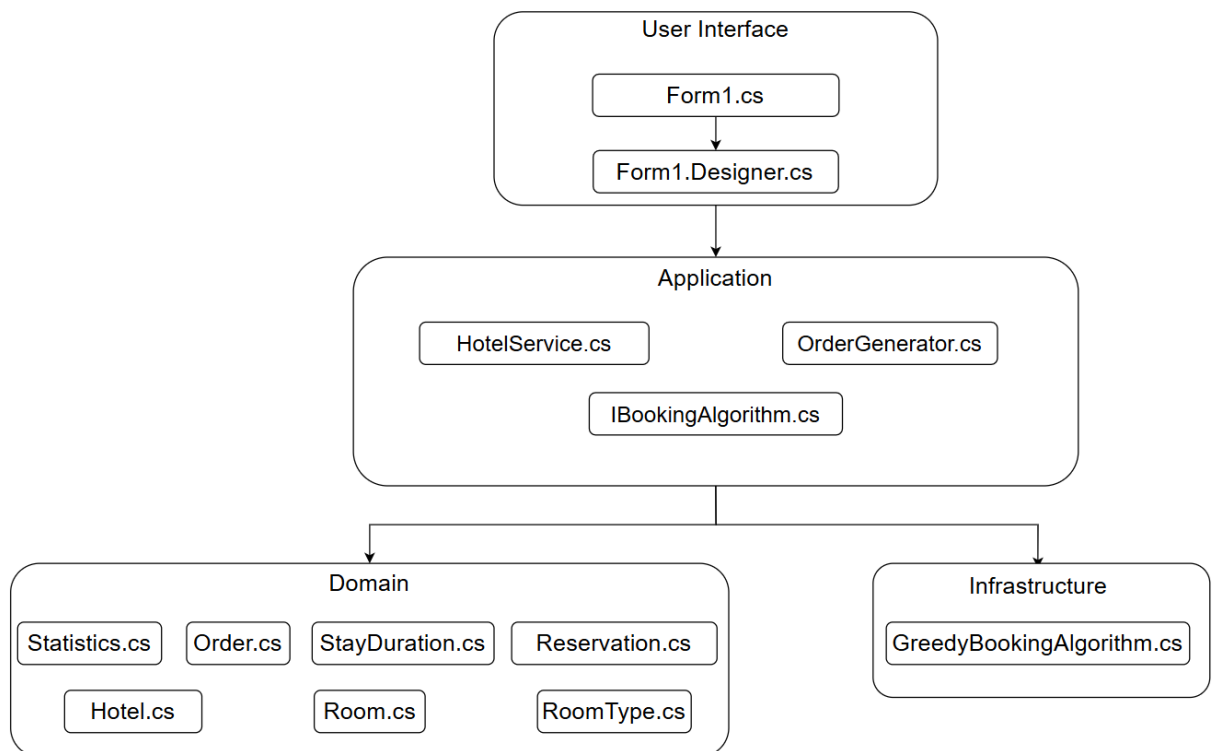


Рисунок - 1 Структурная схема программы

3.1 Доменная область

Классы этого слоя представляют предметную область и не содержат бизнес-логики распределения.

3.1.1 Room

1 Свойства:

- а) `Id (int)` – уникальный идентификатор;
- б) `Type (RoomType)` - тип номера – перечисление `RoomType`;
- в) `PricePerDay (decimal)` - стоимость за сутки;

в) Reservations (List<Reservation>) - список объектов Reservation, связанных с номером.

2 Методы:

а) ToString() - возвращает строку вида "{Id} ({Type})", используется для отладки и отображения;

3.1.2 Reservation

1. Свойства:

- а) OrderId (int) – идентификатор заказа, в рамках которого создано бронирование;
- б) CheckIn (DateTime) – дата заезда;
- в) CheckOut (DateTime) – дата выезда (исключающая, т.е. номер освобождается на следующий день после этой даты);

3.1.3 Order

1. Свойства:

- а) Id (int, только для чтения) – уникальный идентификатор заказа, автоматически увеличивается при создании;
- б) Type (RoomType) – требуемый тип номера;
- в) Duration (StayDuration) – продолжительность проживания (1, 7 или 30 дней);
- г) ArrivalDate (DateTime) – дата поступления заявки;
- д) Deadline (DateTime, вычисляемое, только для чтения) – крайний срок заселения (ArrivalDate.AddDays(3));
- е) IsAssigned (bool) – флаг, показывающий, обслужен ли заказ;
- з) AssignedDate (DateTime?) – дата фактического заселения (если заказ выполнен);
- и) AssignedRoomIds (List<int>) – список идентификаторов номеров, выделенных для данного заказа (поддерживает размещение в нескольких номерах).

2. Методы:

- а) Order() – конструктор по умолчанию – инициализирует Id следующим значением из статического счётчика.

3.1.4 Hotel

1. Свойства:

- а) Rooms (List<Room>) – все номера гостиницы;
- б) Orders (List<Order>) – все заказы, поступившие за всё время моделирования;
- в) NewOrders (List<Order>) – заказы, поступившие в текущий день (для отображения в интерфейсе);
- г) Statistics (Statistics) – объект с агрегированными показателями;
- д) CurrentDay (int) – текущий номер дня симуляции;
- е) LastProcessedDate (DateTime) – календарная дата последнего обработанного дня.

2. Методы:

- а) Hotel() - конструктор создаёт пустые списки, устанавливает CurrentDay = 1, LastProcessedDate = DateTime.Today.AddDays(-1);
- б) InitializeRooms() – заполняет коллекцию Rooms номерами четырёх типов с установленными ценами;
- в) InitializeReservations() – создаёт начальные бронирования для имитации частичной занятости на старте симуляции.

3.1.5 Statistics

1. Свойства:

- а) TotalOrders (int) – общее количество поступивших заказов;
- б) AcceptedOrders (int) – количество успешно обслуженных заказов;
- в) RejectedOrders (int) – количество заказов, по которым истёк срок бронирования;
- г) TotalIncome (decimal) – суммарный доход от всех бронирований.

3.1.1 Вспомогательные перечисления

1 RoomType, определяет допустимые типы номеров:

- а) Single – одноместный;
- б) Double – двухместный;
- в) Triple – трёхместный
- г) Vip – номер повышенной комфортности.

2 StayDuration, задаёт возможную длительность проживания в днях:

- а) Day = 1;
- б) Week = 7;
- в) Month = 30.

3.2 Сервис бизнес логики

Сервисный слой инкапсулирует логику моделирования дня и генерации входящего потока заказов.

3.2.1 IBookingAlgorithm.

1. Методы:

а) void ProcessOrders(List<Order> orders, List<Room> rooms, DateTime today) – интерфейсный метод, выполняющий распределение заказов по номерам.

3.2.2 OrderGenerator

1. Свойства:

а) random (Random) – закрытое поле, генератор случайных чисел.

2. Методы:

а) public List<Order> GenerateOrders(List<Room> rooms, DateTime today) – вычисляет число свободных номеров каждого типа на указанную дату и формирует заказы, количество которых на 1–6 превышает количество свободных мест. Для каждого заказа случайным образом выбирается длительность проживания (1, 7 или 30 дней)..

3.2.3 HotelService

Координирует работу всех компонентов при переходе к следующему дню симуляции. Выступает фасадом для UI.

1. Методы:

а) hotel (Hotel, закрытое) – экземпляр гостиницы;
б) algorithm (IBookingAlgorithm, закрытое) – текущая стратегия распределения;
в) generator (OrderGenerator, закрытое) – генератор заказов;
г) Hotel (Hotel, публичное только для чтения) – предоставляет доступ к объекту гостиницы для отображения на форме.

2. Методы:

а) HotelService() – конструктор, создаёт объект Hotel, инициализирует номера и начальные бронирования, подключает жадный алгоритм и генератор;

б) NextDay() выполняет один шаг моделирования, в ходе которого генерирует и добавляет в список новые заказы, распределяет их по номерам, обновляет финансовую и операционную статистику, а затем увеличивает счетчик дней;

в) private decimal CalculateIncome() – суммирует доход по всем бронированиям (произведение количества дней на цену номера).

3.3 Пользовательский интерфейс

Отвечает за взаимодействие с пользователем, визуализацию данных и обработку событий.

3.3.1 Form1

Наследуется от `System.Windows.Forms.Form`. Содержит все элементы управления и связывает их с сервисом `HotelService`.

1. Свойства:

- а) `hotelService (HotelService)` – ссылка на сервис гостиницы;
- б) `lblDay (Label)` – отображает текущую моделируемую дату;
- в) `btnNextDay (Button)` – кнопка перехода к следующему дню;
- г) `tabControl1 (TabControl)` – контейнер вкладок «Номера», «Заказы», «Статистика»;
- д) `dgvRooms (DataGridView)` – таблица номеров с колонками: Номер, Тип, Цена, Выезд, Осталось дней;
- е) `dgvOrders (DataGridView)` – таблица заказов с колонками: ID, Тип, Длит., Поступил, Дедлайн, Статус, Комнаты;
- з) `dgvStatistics (DataGridView)` – таблица статистики (Всего, Принято, Отклонено, Доход);
- и) `rtbLog (RichTextBox)` – журнал событий (только для чтения).

2. Методы:

- а) `Form1()` – конструктор, инициализирует компоненты формы, создаёт `HotelService`, настраивает сетки и вызывает первичное обновление интерфейса;
- б) `private void ConfigureGrids()` – задаёт структуру колонок для всех `DataGridView`, устанавливает русские заголовки и привязку к свойствам отображаемых объектов;
- в) `UpdateUI()` - обновляет экранную форму, актуализируя метку даты, таблицу статистики и списки заказов с сортировкой по длительности и ID, а также отображает в таблице номеров текущих жильцов, даты их выезда и оставшиеся дни;
- г) `private string GetRoomTypeName(RoomType type)` – вспомогательный метод, возвращает русское название типа номера;
- д) `private void btnNextDay_Click(object sender, EventArgs e)` – обработчик кнопки «Следующий день»: вызывает `hotelService.NextDay()`, обновляет интерфейс и выводит в лог сводку за прошедший день.

3.4 Слой инфраструктуры

В рамках данной учебной задачи специализированные классы инфраструктуры не создавались. Используются стандартные средства платформы `.NET Framework`:

3.4.1 GreedyBookingAlgorithm

1. Методы:

а) `public void ProcessOrders(...)` – реализация жадного алгоритма: сортирует необслуженные заказы по длительности и Id, затем для каждого заказа пытается найти размещение, вызывая вспомогательные методы;

б) `private bool TrySingleRoom(Order order, List<Room> rooms, DateTime start, int duration)` – пытается разместить заказ в одном номере подходящего типа (с учётом повышения);

в) `private bool TryMultipleRooms(Order order, List<Room> rooms, DateTime start, int duration)` – для двух- и трёхместных заказов ищет комбинацию из нескольких номеров меньшей вместимости;

г) `private List<RoomType> GetAllowedTypes(RoomType requested)` – возвращает список типов номеров, доступных для повышения (с учётом бюджетных ограничений);

д) `private bool IsRoomFree(Room room, DateTime start, int duration)` – проверяет, свободен ли номер в заданном интервале

е) `private void BookRoom(Room room, Order order, DateTime start, int duration)` – выполняет бронирование: добавляет Reservation в номер, помечает заказ как выполненный, фиксирует дату заселения и список занятых комнат.

4 Схемы алгоритмов

Ниже приведены описания графических схем, иллюстрирующих ключевые алгоритмы системы бронирования.

Основной алгоритм распределения заказов представлен на рисунке 2. На вход он получает список всех заказов и перечень номеров гостиницы. Вначале из общего списка отбираются необслуженные заказы, чей предельный срок бронирования ещё не истёк. Затем они сортируются по возрастанию длительности проживания (от коротких к длинным), а при равенстве длительности – по идентификатору заказа (что соответствует очередности поступления). Такой порядок гарантирует приоритет клиентам с минимальным сроком проживания: они быстрее освобождают номера, повышая общую заполняемость. Далее для каждого заказа из отсортированного списка предпринимается попытка размещения, начиная с текущего дня и последовательно перебирая следующие дни вплоть до дедлайна. На каждом рассматриваемом дне сначала вызывается вспомогательный алгоритм поиска одного номера (TrySingleRoom). Если он возвращает успех, создаются записи о бронировании, номера помечаются занятыми, а заказ получает статус выполненного. Если одиночный номер подобрать не удалось и заказ предполагает размещение двух или трёх человек (типы Double или Triple), дополнительно вызывается алгоритм комбинирования номеров (TryMultipleRooms). При удачной комбинации номеров заказ также считается обслуженным. Если ни один из способов не дал результата, осуществляется переход к следующей допустимой дате. Когда все даты до дедлайна исчерпаны, а размещение так и не найдено, заказ остаётся в очереди и будет обработан на следующем шаге симуляции (при условии, что дедлайн ещё не наступил). Цикл продолжается, пока не закончатся все активные заказы.

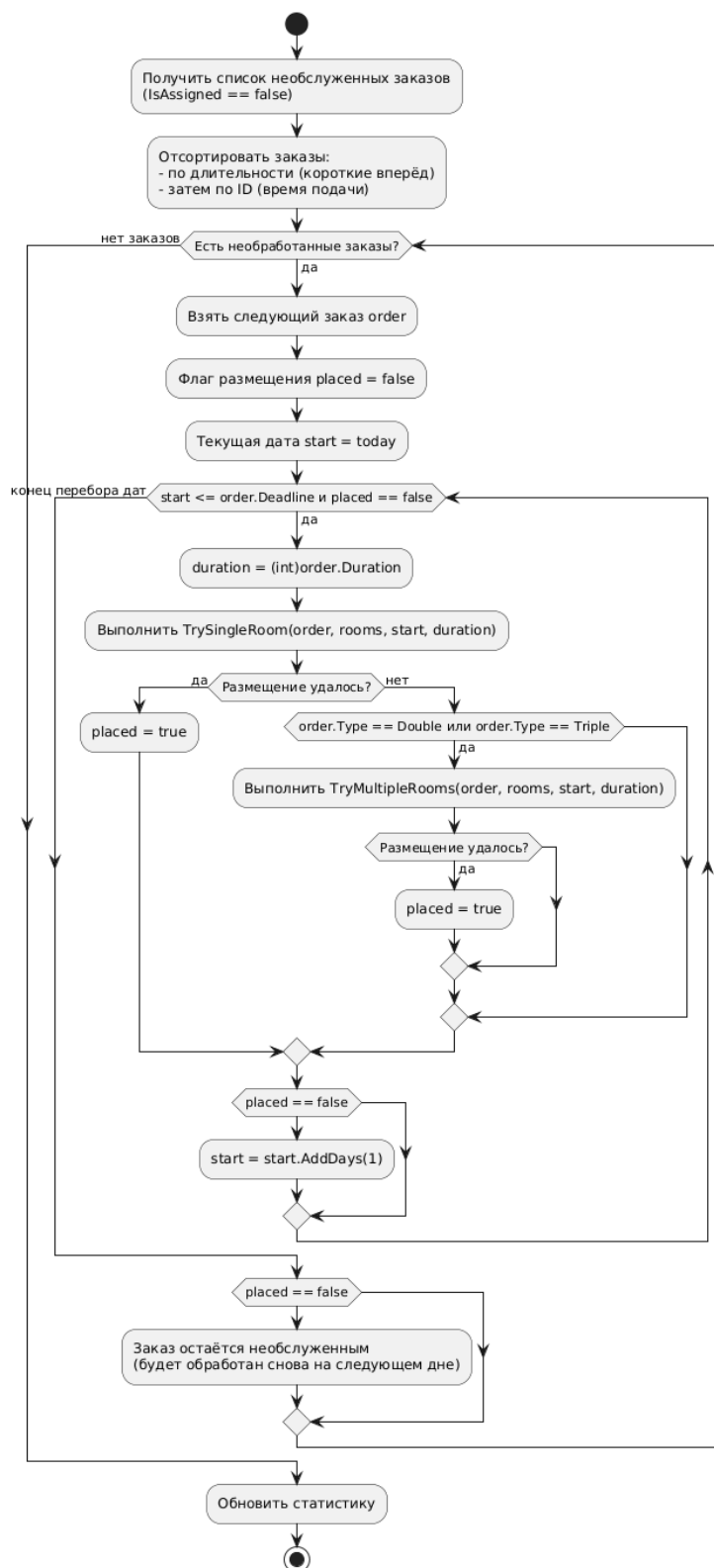


Рисунок 2 - Блок схема основного алгоритма распределения

Алгоритм поиска одного номера (TrySingleRoom) изображён на рисунке 3. Его цель – найти один номер подходящего типа на заданную дату заезда с учётом возможности повышения категории в рамках разумного бюджета. Допустимые типы номеров определяются

по типу заказа: одноместный заказ – одноместный, двухместный; двухместный – двухместный, трёхместный; трёхместный – трёхместный, VIP; VIP – только VIP. Типы перебираются в указанном порядке. Для каждого типа ищется первый свободный номер, у которого в интервале от даты заезда до даты выезда нет пересечений с существующими бронированиями. При обнаружении подходящего номера он немедленно бронируется: в список броней комнаты добавляется новый объект, заказу присваивается статус обслуженного, фиксируются дата заезда и идентификатор занятого номера. Алгоритм возвращает признак успеха. Если ни один из допустимых типов не дал результата, возвращается неудача.

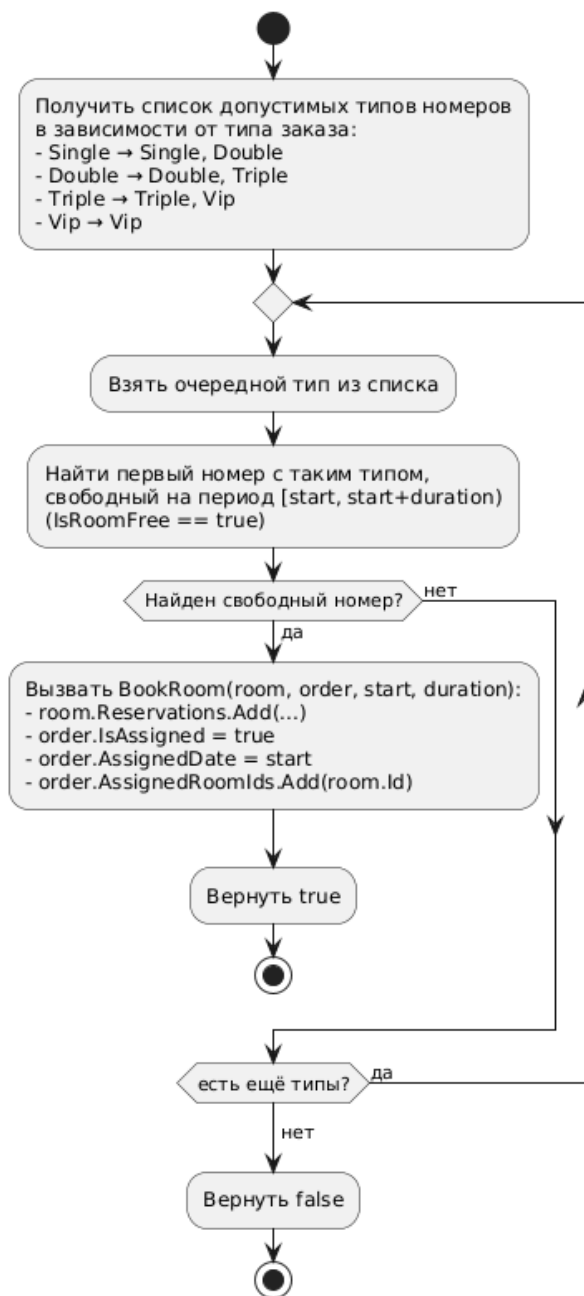


Рисунок 3 – Блок-схема алгоритма TrySingleRoom

Алгоритм комбинирования номеров (TryMultipleRooms) показан на рисунке 4. Он применяется только для заказов типа Double (два человека) и Triple (три человека), когда не удалось найти один номер нужной вместимости. Для двухместного заказа алгоритм пытается найти два свободных одноместных номера на заданный период. Если оба найдены, они бронируются для данного заказа. Для трёхместного заказа реализованы два варианта: сначала ищется пара «одноместный + двухместный»; при неудаче – три одноместных номера. В случае успешного подбора любой из комбинаций все найденные номера бронируются для заказа, а алгоритм возвращает успех. Если ни одна комбинация невозможна, попытка считается неудачной.

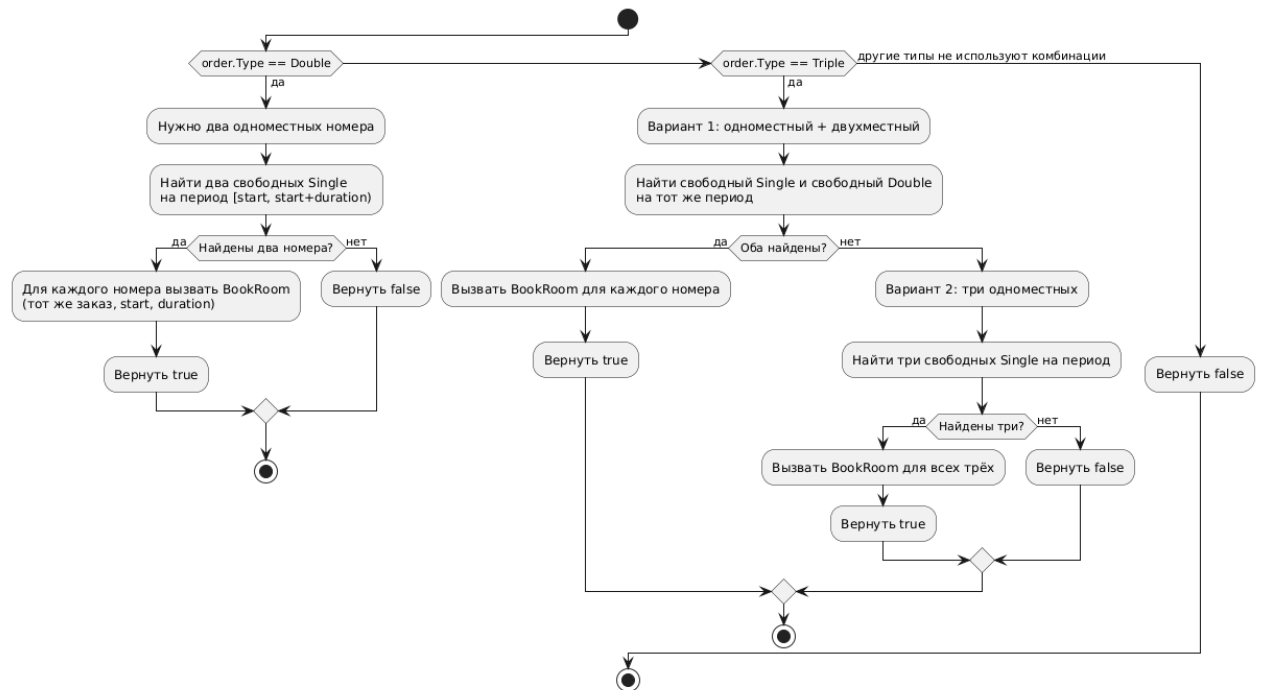


Рисунок 4 – Блок-схема алгоритма TryMultipleRooms

5 Результаты тестирования программы на различных наборах данных

Тестирование проводилось в строгом соответствии с условием задачи: генератор заявок всегда создаёт избыточный спрос – количество запросов каждого типа превышает число свободных номеров этого типа в текущий день.

5.1 Тест 1 – с равномерным избыточным спросом

На рисунке 5 показаны начальные условия: отель загружен по умолчанию. Генератор на каждом шагу создаёт для каждого типа заявок больше, чем свободных номеров этого типа.

Дата: 01.07.2026

Следующий день

Номера	Заказы	Статистика	Цена	Выезд	Осталось
▶	1	Одноместный	3000	03.07.2026	3
	2	Одноместный	3000	03.07.2026	3
	3	Одноместный	3000	03.07.2026	3
	4	Одноместный	3000	Свободен	0
	5	Одноместный	3000	Свободен	0
	6	Одноместный	3000	Свободен	0
	7	Одноместный	3000	Свободен	0
	8	Одноместный	3000	Свободен	0
	9	Одноместный	3000	Свободен	0
	10	Одноместный	3000	Свободен	0
	11	Одноместный	3000	Свободен	0
	12	Одноместный	3000	Свободен	0
	13	Одноместный	3000	Свободен	0
	14	Одноместный	3000	Свободен	0
	15	Одноместный	3000	Свободен	0
	16	Одноместный	3000	Свободен	0
	17	Одноместный	3000	Свободен	0
	18	Одноместный	3000	Свободен	0
	19	Одноместный	3000	Свободен	0
	20	Одноместный	3000	Свободен	0
	21	Одноместный	3000	Свободен	0
	22	Одноместный	3000	Свободен	0
	23	Одноместный	3000	Свободен	0
	24	Одноместный	3000	Свободен	0
	25	Одноместный	3000	Свободен	0
	26	Одноместный	3000	Свободен	0
	27	Одноместный	3000	Свободен	0

Рисунок 5 – начальные условия теста 1

На рисунке 6 изображенна вкладка с номерами на 5 день симуляции. Результатом является полное заселение всех номеров.

Дата: 06.07.2026

Следующий день

Номера	Заказы	Статистика		
Номер	Тип	Цена	Выезд	Осталось
1	Одноместный	3000	10.07.2026	5
2	Одноместный	3000	02.08.2026	28
3	Одноместный	3000	02.08.2026	28
4	Одноместный	3000	10.07.2026	5
5	Одноместный	3000	09.07.2026	4
6	Одноместный	3000	09.07.2026	4
7	Одноместный	3000	09.07.2026	4
8	Одноместный	3000	09.07.2026	4
9	Одноместный	3000	08.07.2026	3
10	Одноместный	3000	08.07.2026	3
11	Одноместный	3000	08.07.2026	3
12	Одноместный	3000	08.07.2026	3
13	Одноместный	3000	08.07.2026	3
14	Одноместный	3000	08.07.2026	3
15	Одноместный	3000	08.07.2026	3
16	Одноместный	3000	08.07.2026	3
17	Одноместный	3000	08.07.2026	3
18	Одноместный	3000	08.07.2026	3
19	Одноместный	3000	08.07.2026	3
20	Одноместный	3000	08.07.2026	3
21	Одноместный	3000	31.07.2026	26
22	Одноместный	3000	31.07.2026	26
23	Одноместный	3000	31.07.2026	26
24	Одноместный	3000	31.07.2026	26
25	Одноместный	3000	31.07.2026	26
26	Одноместный	3000	31.07.2026	26
27	Одноместный	3000	31.07.2026	26

Дата: 01.07.2026
Всего заказов: 67
Полнота: 66
Отклонено: 0
Доход: 5720000

Дата: 02.07.2026
Всего заказов: 84
Полнота: 82
Отклонено: 0
Доход: 6720000

Дата: 03.07.2026
Всего заказов: 96
Полнота: 88
Отклонено: 0
Доход: 7720000

Рисунок 6 – Работа программы на 5 день симуляции

На рисунке 7 представленна таблица с ожидающими заселения заказами. Таблица постоянно содержит десятки заявок, у большинства оставшийся срок до дедлайна равен 2-3 дням – заявки успевают обслужиться.

Номера		Заказы	Статистика				
	ID	Тип	Длит.	Поступил	Дедлайн	Статус	Комнаты
▶	88	Двухмест...	1	03.07.2026	06.07.2026	Ожидает	—
	91	Одномест...	1	04.07.2026	07.07.2026	Ожидает	—
	93	Трёхмест...	1	04.07.2026	07.07.2026	Ожидает	—
	89	Трёхмест...	7	03.07.2026	06.07.2026	Ожидает	—
	90	VIP	7	03.07.2026	06.07.2026	Ожидает	—
	92	Двухмест...	7	04.07.2026	07.07.2026	Ожидает	—
	85	Одномест...	30	03.07.2026	06.07.2026	Ожидает	—
	86	Одномест...	30	03.07.2026	06.07.2026	Ожидает	—
	94	VIP	30	04.07.2026	07.07.2026	Ожидает	—

Рисунок 6 – Таблица ожидающих заселения на 7 день симуляции.

На рисунке 8 представлена таблица с уже заселенными заказами, большинство номеров заселено с маленьким сроком проживания, что подтверждает описанный выше приоритет и позволяет обслужить большое количество заказов.

Номера Заказы Статистика						
ID	Тип	Дат.	Поступил	Дедлайн	Статус	Комнаты
2	Одномест...	1	01.07.2026	04.07.2026	Заселён	4
3	Одномест...	1	01.07.2026	04.07.2026	Заселён	5
5	Одномест...	1	01.07.2026	04.07.2026	Заселён	6
6	Одномест...	1	01.07.2026	04.07.2026	Заселён	7
12	Одномест...	1	01.07.2026	04.07.2026	Заселён	8
14	Одномест...	1	01.07.2026	04.07.2026	Заселён	9
15	Одномест...	1	01.07.2026	04.07.2026	Заселён	10
17	Одномест...	1	01.07.2026	04.07.2026	Заселён	11
18	Одномест...	1	01.07.2026	04.07.2026	Заселён	12
21	Одномест...	1	01.07.2026	04.07.2026	Заселён	13
22	Одномест...	1	01.07.2026	04.07.2026	Заселён	14
25	Одномест...	1	01.07.2026	04.07.2026	Заселён	15
28	Одномест...	1	01.07.2026	04.07.2026	Заселён	16
31	Двумест...	1	01.07.2026	04.07.2026	Заселён	33
32	Двумест...	1	01.07.2026	04.07.2026	Заселён	34
33	Двумест...	1	01.07.2026	04.07.2026	Заселён	35
34	Двумест...	1	01.07.2026	04.07.2026	Заселён	36
37	Двумест...	1	01.07.2026	04.07.2026	Заселён	37
38	Двумест...	1	01.07.2026	04.07.2026	Заселён	38
41	Двумест...	1	01.07.2026	04.07.2026	Заселён	39
44	Двумест...	1	01.07.2026	04.07.2026	Заселён	40
52	Трёхмест...	1	01.07.2026	04.07.2026	Заселён	51
56	Трёхмест...	1	01.07.2026	04.07.2026	Заселён	52
59	Трёхмест...	1	01.07.2026	04.07.2026	Заселён	53
61	Трёхмест...	1	01.07.2026	04.07.2026	Заселён	54
64	Трёхмест...	1	01.07.2026	04.07.2026	Заселён	55

Рисунок 8 – Таблица с заселенными заказами на 5 день симуляции

5.2 Тест 2 - Повышенный спрос на трехместные номера и пониженный на остальные

На рисунке 9 изображены начальные условия: отель загружен по умолчанию. Генератор на каждом шагу создаёт для каждого типа заявок больше, чем свободных номеров этого типа. На трехместные номера повышенный спрос.

Дата: 01.07.2026

Следующий день

Номера	Заказы	Статистика		
Номер	Тип	Цена	Выезд	Осталось
1	Одноместный	3000	03.07.2026	3
2	Одноместный	3000	03.07.2026	3
3	Одноместный	3000	03.07.2026	3
4	Одноместный	3000	Свободен	0
5	Одноместный	3000	Свободен	0
6	Одноместный	3000	Свободен	0
7	Одноместный	3000	Свободен	0
8	Одноместный	3000	Свободен	0
9	Одноместный	3000	Свободен	0
10	Одноместный	3000	Свободен	0
11	Одноместный	3000	Свободен	0
12	Одноместный	3000	Свободен	0
13	Одноместный	3000	Свободен	0
14	Одноместный	3000	Свободен	0
15	Одноместный	3000	Свободен	0
16	Одноместный	3000	Свободен	0
17	Одноместный	3000	Свободен	0
18	Одноместный	3000	Свободен	0
19	Одноместный	3000	Свободен	0
20	Одноместный	3000	Свободен	0
21	Одноместный	3000	Свободен	0
22	Одноместный	3000	Свободен	0
23	Одноместный	3000	Свободен	0
24	Одноместный	3000	Свободен	0
25	Одноместный	3000	Свободен	0
26	Одноместный	3000	Свободен	0
27	Одноместный	3000	Свободен	0

Рисунок 9 - начальные условия второго теста

На рисунке 10 показана вкладка с номерами на 8 день симуляции. Результатом так же является полное заселение всех номеров.

Бронирование гостиницы

Дата: 09.07.2026

Следующий день

Номера	Заказы	Статистика		
Номер	Тип	Цена	Выезд	Осталось
1	Одноместный	3000	04.08.2026	27
2	Одноместный	3000	09.07.2026	1
3	Одноместный	3000	10.07.2026	2
4	Одноместный	3000	02.08.2026	25
5	Одноместный	3000	02.08.2026	25
6	Одноместный	3000	02.08.2026	25
7	Одноместный	3000	09.07.2026	1
8	Одноместный	3000	09.07.2026	1
9	Одноместный	3000	09.07.2026	1
10	Одноместный	3000	01.08.2026	24
11	Одноместный	3000	01.08.2026	24
12	Одноместный	3000	01.08.2026	24
13	Одноместный	3000	09.07.2026	1
14	Одноместный	3000	09.07.2026	1
15	Одноместный	3000	09.07.2026	1
16	Одноместный	3000	15.07.2026	7
17	Одноместный	3000	15.07.2026	7
18	Одноместный	3000	15.07.2026	7
19	Одноместный	3000	07.08.2026	30
20	Одноместный	3000	07.08.2026	30
21	Одноместный	3000	07.08.2026	30
22	Одноместный	3000	31.07.2026	23
23	Одноместный	3000	31.07.2026	23
24	Одноместный	3000	31.07.2026	23
25	Одноместный	3000	31.07.2026	23
26	Одноместный	3000	31.07.2026	23
27	Одноместный	3000	31.07.2026	23

Рисунок 10 - Работа программы на 8 день симуляции второго теста

Рисунок 11 подтверждает повышенный спрос на трехместные номера. По сравнению с первым тестом количество ожидающих заказов возросло.

Дата: 09.07.2026 Следующий день

Номера Заказы Статистика

	ID	Тип	Длит.	Поступил	Дедлайн	Статус	Комнаты
	103	VIP	30	08.07.2026	11.07.2026	Ожидает	—
	90	Одномест...	1	08.07.2026	11.07.2026	Заселён	6
	91	Одномест...	1	08.07.2026	11.07.2026	Заселён	7
	94	Одномест...	1	08.07.2026	11.07.2026	Заселён	8
	97	Трёхмест...	1	08.07.2026	11.07.2026	Заселён	54
	98	Трёхмест...	1	08.07.2026	11.07.2026	Заселён	55
	99	Трёхмест...	1	08.07.2026	11.07.2026	Заселён	56
	100	Трёхмест...	1	08.07.2026	11.07.2026	Заселён	57
	83	Одномест...	7	08.07.2026	11.07.2026	Заселён	9
	88	Одномест...	7	08.07.2026	11.07.2026	Заселён	10
	89	Одномест...	7	08.07.2026	11.07.2026	Заселён	11
	92	Одномест...	7	08.07.2026	11.07.2026	Заселён	12
	95	Двухмест...	7	08.07.2026	11.07.2026	Заселён	31
	96	Трёхмест...	7	08.07.2026	11.07.2026	Заселён	58
	84	Одномест...	30	08.07.2026	11.07.2026	Заселён	13
	85	Одномест...	30	08.07.2026	11.07.2026	Заселён	14

Рисунок 11 - Таблица ожидающих заселения на 8 день симуляции второго теста.

На рисунке 12 показана таблица с уже заселенными заказами, большинство номеров, как и в первом тесте, заселено с маленьким сроком проживания, что еще раз подтверждает описанный выше приоритет и позволяет обслужить большое количество заказов.

Дата: 09.07.2026 Следующий день

Номера Заказы Статистика

	Номер	Тип	Цена	Выезд	Осталось
	49	Двухместный	5000	06.08.2026	29
	50	Двухместный	5000	Свободен	0
	51	Трёхместный	7000	09.07.2026	1
	52	Трёхместный	7000	09.07.2026	1
	53	Трёхместный	7000	09.07.2026	1
	54	Трёхместный	7000	09.07.2026	1
	55	Трёхместный	7000	09.07.2026	1
	56	Трёхместный	7000	09.07.2026	1
	57	Трёхместный	7000	09.07.2026	1
	58	Трёхместный	7000	15.07.2026	7
	59	Трёхместный	7000	31.07.2026	23
	60	Трёхместный	7000	31.07.2026	23
	61	VIP	15000	31.07.2026	23
	62	VIP	15000	02.08.2026	25
	63	VIP	15000	02.08.2026	25
	64	VIP	15000	01.08.2026	24

Рисунок 12 - Таблица с заселенными заказами на 8 день симуляции второго теста

Список используемых источников

- 1 Мокроусов, М. Н. Методические указания по оформлению отчётов и пояснительных записок для студентов, обучающихся по направлению 09.03.01 «Информатика и вычислительная техника», профили «Автоматизированные системы обработки информации и управления», «Системы автоматизированного проектирования», очная и заочная формы обучения / М. Н. Мокроусов, Н. В. Соболева. – Ижевск : Изд-во ИжГТУ имени М. Т. Калашникова, 2024. – 21 с
- 2 Обзор Entity Framework Core // Microsoft Learn : сайт. – URL: <https://learn.microsoft.com/ru-ru/ef/core/> (дата обращения: 29.06.2026)
- 3 Онлайн-сервис для создания диаграмм draw.io // Diagrams.net : сайт.– URL: <https://app.diagrams.net/> (дата обращения: 29.06.2026).
- 4 Интерактивные образовательные курсы Ulearn.me // Учи.ру : сайт.
URL: <https://ulearn.me/> (дата обращения: 29.06.2026).
- 5 Онлайн библиотека // wikipedia : сайт.
URL: <https://ru.wikipedia.org/> (дата обращения: 29.06.2026).

1 Директория Models

1.1 Файл Hotel.cs:

```
using HotelBooking.Enums;
using System;
using System.Collections.Generic;

namespace HotelBooking.Models
{
    public class Hotel
    {
        public List<Room> Rooms { get; set; }
        public List<Order> Orders { get; set; }
        public List<Order> NewOrders { get; set; }
        public Statistics Statistics { get; set; }
        public int CurrentDay { get; set; }
        public DateTime LastProcessedDate { get; set; }

        public Hotel()
        {
            Rooms = new List<Room>();
            Orders = new List<Order>();
            NewOrders = new List<Order>();
            Statistics = new Statistics();
            CurrentDay = 1;
            LastProcessedDate = DateTime.Today.AddDays(-1);
        }

        public void InitializeRooms()
        {
            int id = 1;
            for (int i = 0; i < 30; i++)
                Rooms.Add(new Room { Id = id++, Type = RoomType.Single,
PricePerDay = 3000 });
            for (int i = 0; i < 20; i++)
                Rooms.Add(new Room { Id = id++, Type = RoomType.Double,
PricePerDay = 5000 });
            for (int i = 0; i < 10; i++)
                Rooms.Add(new Room { Id = id++, Type = RoomType.Triple,
PricePerDay = 7000 });
            for (int i = 0; i < 5; i++)
                Rooms.Add(new Room { Id = id++, Type = RoomType.Vip,
PricePerDay = 15000 });
        }

        public void InitializeReservations()
        {
            DateTime today = DateTime.Today;
            for (int i = 0; i < 3; i++)
                Rooms[i].Reservations.Add(new Reservation
                {
                    OrderId = 0,
                    CheckIn = today,
                    CheckOut = today.AddDays(2)
                });
            for (int i = 30; i < 32; i++)
                Rooms[i].Reservations.Add(new Reservation
                {
                    OrderId = 0,
                    CheckIn = today,
```

```

        CheckOut = today.AddDays(7)
    });
    Rooms[60].Reservations.Add(new Reservation
    {
        OrderId = 0,
        CheckIn = today,
        CheckOut = today.AddDays(30)
    });
    }
}

```

1.2 Файл Order.cs:

```

using HotelBooking.Enums;
using System;
using System.Collections.Generic;

namespace HotelBooking.Models
{
    public class Order
    {
        private static int nextId = 1;

        public int Id { get; }
        public RoomType Type { get; set; }
        public StayDuration Duration { get; set; }
        public DateTime ArrivalDate { get; set; }

        public DateTime Deadline => ArrivalDate.AddDays(3);

        public bool IsAssigned { get; set; }
        public DateTime? AssignedDate { get; set; }
        public List<int> AssignedRoomIds { get; set; } = new List<int>();

        public Order()
        {
            Id = nextId++;
        }
    }
}

```

1.3 Файл Reservation.cs:

```

using System;

namespace HotelBooking.Models
{
    public class Reservation
    {
        public int OrderId { get; set; }

        public DateTime CheckIn { get; set; }

        public DateTime CheckOut { get; set; }
    }
}

```

1.4 Файл Room.cs:

```

using HotelBooking.Enums;
using System.Collections.Generic;

namespace HotelBooking.Models
{
    public class Room
    {
        public int Id { get; set; }
    }
}

```

```

        public RoomType Type { get; set; }

        public decimal PricePerDay { get; set; }

        public List<Reservation> Reservations { get; set; }
            = new List<Reservation>();

        public override string ToString()
        {
            return $"{Id} ({Type})";
        }
    }
}

```

1.5 Файл Statistics.cs:

```

namespace HotelBooking.Models
{
    public class Statistics
    {
        public int TotalOrders { get; set; }

        public int AcceptedOrders { get; set; }

        public int RejectedOrders { get; set; }

        public decimal TotalIncome { get; set; }
    }
}

```

2 Директория Services

2.1 Файл HotelService.cs:

```

using HotelBooking.Algorithms;
using HotelBooking.Models;
using System;
using System.Linq;

namespace HotelBooking.Services
{
    public class HotelService
    {
        private readonly Hotel hotel;
        private readonly IBookingAlgorithm algorithm;
        private readonly OrderGenerator generator;

        public Hotel Hotel => hotel;

        public HotelService()
        {
            hotel = new Hotel();
            hotel.InitializeRooms();
            hotel.InitializeReservations();

            algorithm = new GreedyBookingAlgorithm();
            generator = new OrderGenerator();
        }

        public void NextDay()
        {
            DateTime today = DateTime.Today.AddDays(hotel.CurrentDay - 1);

            var newOrders = generator.GenerateOrders(hotel.Rooms, today);
        }
    }
}

```

```

        hotel.NewOrders = newOrders;
        hotel.Orders.AddRange(newOrders);

        hotel.Statistics.TotalOrders += newOrders.Count;

        algorithm.ProcessOrders(hotel.Orders, hotel.Rooms, today);

        hotel.LastProcessedDate = today;

        hotel.Statistics.AcceptedOrders = hotel.Orders.Count(o =>
o.IsAssigned);
        hotel.Statistics.RejectedOrders = hotel.Orders.Count(o =>
!o.IsAssigned && o.Deadline < today);
        hotel.Statistics.TotalIncome = CalculateIncome();

        hotel.CurrentDay++;
    }

    private decimal CalculateIncome()
    {
        decimal income = 0;
        foreach (var room in hotel.Rooms)
        {
            foreach (var res in room.Reservations)
            {
                int days = (res.CheckOut - res.CheckIn).Days;
                income += days * room.PricePerDay;
            }
        }
        return income;
    }
}
}

```

2.2 Файл OrderGenerator.cs:

```

using HotelBooking.Enums;
using HotelBooking.Models;
using System;
using System.Collections.Generic;
using System.Linq;

namespace HotelBooking.Services
{
    public class OrderGenerator
    {
        private Random random = new Random();

        public List<Order> GenerateOrders(List<Room> rooms, DateTime today)
        {
            List<Order> orders = new List<Order>();

            var freeCounts = new Dictionary<RoomType, int>();
            foreach (RoomType type in Enum.GetValues(typeof(RoomType)))
            {
                int free = rooms.Count(r =>
                    r.Type == type &&
                    !r.Reservations.Any(res =>
                        res.CheckIn <= today && res.CheckOut > today));
                freeCounts[type] = free;
            }

            foreach (RoomType type in Enum.GetValues(typeof(RoomType)))
            {
                int free = freeCounts[type];
            }
        }
    }
}

```



```

        if (free == 0)
            continue;

        int excess = random.Next(1, 7);
        int orderCount = free + excess;

        for (int i = 0; i < orderCount; i++)
        {
            StayDuration duration;
            int val = random.Next(3);
            switch (val)
            {
                case 0: duration = StayDuration.Day; break;
                case 1: duration = StayDuration.Week; break;
                default: duration = StayDuration.Month; break;
            }

            orders.Add(new Order
            {
                Type = type,
                Duration = duration,
                ArrivalDate = today
            });
        }
    }

    return orders;
}
}
}

```

3 Директория Enums

3.1 Файл RoomType.cs:

```

namespace HotelBooking.Enums
{
    public enum RoomType
    {
        Single,
        Double,
        Triple,
        Vip
    }
}

```

3.2 Файл StayDuration.cs:

```

namespace HotelBooking.Enums
{
    public enum StayDuration
    {
        Day = 1,
        Week = 7,
        Month = 30
    }
}

```

4 Директория Algorithms

4.1 Файл GreedyBookingAlgorithm.cs:

```

using System;
using System.Collections.Generic;
using System.Linq;
using HotelBooking.Enums;
using HotelBooking.Models;

```

```

namespace HotelBooking.Algorithms
{
    public class GreedyBookingAlgorithm : IBookingAlgorithm
    {
        public void ProcessOrders(List<Order> orders, List<Room> rooms,
DateTime today)
        {
            var sortedOrders = orders
                .Where(o => !o.IsAssigned)
                .OrderBy(o => (int)o.Duration)
                .ThenBy(o => o.Id)
                .ToList();

            foreach (var order in sortedOrders)
            {
                bool placed = false;

                for (DateTime start = today; start <= order.Deadline &&
!placed; start = start.AddDays(1))
                {
                    int duration = (int)order.Duration;

                    placed = TrySingleRoom(order, rooms, start, duration);
                    if (placed) break;

                    if (!placed && (order.Type == RoomType.Double ||
order.Type == RoomType.Triple))
                    {
                        placed = TryMultipleRooms(order, rooms, start,
duration);
                        if (placed) break;
                    }
                }
            }

            private bool TrySingleRoom(Order order, List<Room> rooms, DateTime
start, int duration)
            {
                var allowedTypes = GetAllowedTypes(order.Type);
                foreach (var type in allowedTypes)
                {
                    var freeRoom = rooms
                        .Where(r => r.Type == type)
                        .FirstOrDefault(r => IsRoomFree(r, start, duration));
                    if (freeRoom != null)
                    {
                        BookRoom(freeRoom, order, start, duration);
                        return true;
                    }
                }
                return false;
            }

            private bool TryMultipleRooms(Order order, List<Room> rooms,
DateTime start, int duration)
            {
                if (order.Type == RoomType.Double)
                {
                    var singleRooms = rooms
                        .Where(r => r.Type == RoomType.Single && IsRoomFree(r,
start, duration))
                        .Take(2)
                        .ToList();

```

```

        if (singleRooms.Count == 2)
        {
            foreach (var r in singleRooms)
                BookRoom(r, order, start, duration);
            return true;
        }
    }
    else if (order.Type == RoomType.Triple)
    {
        var single = rooms
            .FirstOrDefault(r => r.Type == RoomType.Single &&
                IsRoomFree(r, start, duration));
        var doubleRoom = rooms
            .FirstOrDefault(r => r.Type == RoomType.Double &&
                IsRoomFree(r, start, duration));
        if (single != null && doubleRoom != null)
        {
            BookRoom(single, order, start, duration);
            BookRoom(doubleRoom, order, start, duration);
            return true;
        }

        var threeSingles = rooms
            .Where(r => r.Type == RoomType.Single && IsRoomFree(r,
start, duration))
            .Take(3)
            .ToList();
        if (threeSingles.Count == 3)
        {
            foreach (var r in threeSingles)
                BookRoom(r, order, start, duration);
            return true;
        }
    }
    return false;
}

private List<RoomType> GetAllowedTypes(RoomType requested)
{
    switch (requested)
    {
        case RoomType.Single:
            return new List<RoomType> { RoomType.Single,
RoomType.Double };
        case RoomType.Double:
            return new List<RoomType> { RoomType.Double,
RoomType.Triple };
        case RoomType.Triple:
            return new List<RoomType> { RoomType.Triple,
RoomType.Vip };
        case RoomType.Vip:
            return new List<RoomType> { RoomType.Vip };
        default:
            return new List<RoomType> { requested };
    }
}

private bool IsRoomFree(Room room, DateTime start, int duration)
{
    DateTime end = start.AddDays(duration);
    foreach (var res in room.Reservations)
    {
        if (start < res.CheckOut && end > res.CheckIn)
            return false;
    }
}

```

```

        return true;
    }

    private void BookRoom(Room room, Order order, DateTime start, int
duration)
    {
        room.Reservations.Add(new Reservation
        {
            OrderId = order.Id,
            CheckIn = start,
            CheckOut = start.AddDays(duration)
        });
        order.IsAssigned = true;
        order.AssignedDate = start;
        order.AssignedRoomIds.Add(room.Id);
    }
}

```

4.2 Файл IBookingAlgorithm.cs:

```

using System;
using System.Collections.Generic;
using HotelBooking.Models;

namespace HotelBooking.Algorithms
{
    public interface IBookingAlgorithm
    {
        void ProcessOrders(
            List<Order> orders,
            List<Room> rooms,
            DateTime today);
    }
}

```

5 Директория UI

5.1 Файл Form1.cs:

```

using System;
using System.Linq;
using System.Windows.Forms;
using HotelBooking.Enums;
using HotelBooking.Models;
using HotelBooking.Services;

namespace HotelBooking
{
    public partial class Form1 : Form
    {
        private HotelService hotelService;

        public Form1()
        {
            InitializeComponent();
            hotelService = new HotelService();
            ConfigureGrids();
            UpdateUI();
        }

        private string GetRoomTypeName(RoomType type)
        {
            switch (type)
            {
                case RoomType.Single: return "Одноместный";
                case RoomType.Double: return "Двухместный";
            }
        }
    }
}

```

```

        case RoomType.Triple: return "Трёхместный";
        case RoomType.Vip: return "VIP";
        default: return type.ToString();
    }
}

private void ConfigureGrids()
{
    dgvRooms.AutoGenerateColumns = false;
    dgvRooms.Columns.Clear();
    dgvRooms.Columns.Add(new DataGridViewTextBoxColumn { HeaderText
= "Номер", DataPropertyName = "Id", Width = 50 });
    dgvRooms.Columns.Add(new DataGridViewTextBoxColumn { HeaderText
= "Тип", DataPropertyName = "Type", Width = 80 });
    dgvRooms.Columns.Add(new DataGridViewTextBoxColumn { HeaderText
= "Цена", DataPropertyName = "Price", Width = 60 });
    dgvRooms.Columns.Add(new DataGridViewTextBoxColumn { HeaderText
= "Выезд", DataPropertyName = "CheckOutDate", Width = 80 });
    dgvRooms.Columns.Add(new DataGridViewTextBoxColumn { HeaderText
= "Осталось дней", DataPropertyName = "DaysLeft", Width = 80 });

    dgvOrders.AutoGenerateColumns = false;
    dgvOrders.Columns.Clear();
    dgvOrders.Columns.Add(new DataGridViewTextBoxColumn { HeaderText
= "ID", DataPropertyName = "Id", Width = 40 });
    dgvOrders.Columns.Add(new DataGridViewTextBoxColumn { HeaderText
= "Тип", DataPropertyName = "Type", Width = 70 });
    dgvOrders.Columns.Add(new DataGridViewTextBoxColumn { HeaderText
= "Длит.", DataPropertyName = "Duration", Width = 50 });
    dgvOrders.Columns.Add(new DataGridViewTextBoxColumn { HeaderText
= "Поступил", DataPropertyName = "Arrival", Width = 80 });
    dgvOrders.Columns.Add(new DataGridViewTextBoxColumn { HeaderText
= "Дедлайн", DataPropertyName = "Deadline", Width = 80 });
    dgvOrders.Columns.Add(new DataGridViewTextBoxColumn { HeaderText
= "Статус", DataPropertyName = "Status", Width = 80 });
    dgvOrders.Columns.Add(new DataGridViewTextBoxColumn { HeaderText
= "Комнаты", DataPropertyName = "Room", Width = 100 });

    dgvStatistics.AutoGenerateColumns = true;
}

private void UpdateUI()
{
    var hotel = hotelService.Hotel;
    DateTime today = DateTime.Today.AddDays(hotel.CurrentDay - 1);
    DateTime lastProcessed = hotel.LastProcessedDate;

    lblDay.Text = $"Дата: {today:dd.MM.yyyy}";

    dgvRooms.DataSource = hotel.Rooms.Select(r =>
    {
        var active = r.Reservations.FirstOrDefault(res =>
res.CheckIn <= today && res.CheckOut >= today);
        string checkOutStr = active == null ? "Свободен" :
active.CheckOut.ToShortDateString();
        int daysLeft = active == null ? 0 : (active.CheckOut -
today).Days + 1;
        return new
        {
            Id = r.Id,
            Type = GetRoomTypeName(r.Type),
            Price = r.PricePerDay,
            CheckOutDate = checkOutStr,
            DaysLeft = daysLeft
        };
    });
}

```

```

    }).ToList();

    var activeOrders = hotel.Orders
        .Where(o => !o.IsAssigned && o.Deadline.Date >= today.Date)
        .OrderBy(o => (int)o.Duration)
        .ThenBy(o => o.Id)
        .Select(o => new
        {
            Id = o.Id,
            Type = GetRoomTypeName(o.Type),
            Duration = (int)o.Duration,
            Arrival = o.ArrivalDate.ToShortDateString(),
            Deadline = o.Deadline.ToShortDateString(),
            Status = "Ожидает",
            Room = "-"
        });

    var assignedToday = hotel.Orders
        .Where(o => o.IsAssigned && o.AssignedDate.HasValue &&
o.AssignedDate.Value.Date == lastProcessed.Date)
        .OrderBy(o => (int)o.Duration)
        .ThenBy(o => o.Id)
        .Select(o =>
        {
            string roomsStr = o.AssignedRoomIds.Count > 0
                ? string.Join(", ", o.AssignedRoomIds)
                : "?";
            return new
            {
                Id = o.Id,
                Type = GetRoomTypeName(o.Type),
                Duration = (int)o.Duration,
                Arrival = o.ArrivalDate.ToShortDateString(),
                Deadline = o.Deadline.ToShortDateString(),
                Status = "Заселён",
                Room = roomsStr
            };
        });

    var allDisplayOrders =
activeOrders.Concat(assignedToday).ToList();
    dgvOrders.DataSource = allDisplayOrders;

    dgvStatistics.DataSource = new[]
    {
        new
        {
            Всего = hotel.Statistics.TotalOrders,
            Принято = hotel.Statistics.AcceptedOrders,
            Отклонено = hotel.Statistics.RejectedOrders,
            Доход = hotel.Statistics.TotalIncome
        }
    };
}

private void btnNextDay_Click(object sender, EventArgs e)
{
    hotelService.NextDay();
    UpdateUI();

    var h = hotelService.Hotel;
    DateTime yesterday = h.LastProcessedDate;
    rtbLog.AppendText($"Дата: {yesterday:dd.MM.yyyy}\n");
    rtbLog.AppendText($"Всего заказов:
{h.Statistics.TotalOrders}\n");

```

```

        rtbLog.AppendText($"Принято: {h.Statistics.AcceptedOrders}\n");
        rtbLog.AppendText($"Отклонено:
{h.Statistics.RejectedOrders}\n");
        rtbLog.AppendText($"Доход: {h.Statistics.TotalIncome}\n");
        rtbLog.AppendText("-----\n");
    }
}
}

```

5.2 Файл Form1.Designer.cs:

```

namespace HotelBooking
{
    partial class Form1
    {
        private System.ComponentModel.IContainer components = null;

        private System.Windows.Forms.Label lblDay;
        private System.Windows.Forms.Button btnNextDay;

        private System.Windows.Forms.TabControl tabControl1;
        private System.Windows.Forms.TabPage tabRooms;
        private System.Windows.Forms.TabPage tabOrders;
        private System.Windows.Forms.TabPage tabStats;

        private System.Windows.Forms.DataGridView dgvRooms;
        private System.Windows.Forms.DataGridView dgvOrders;
        private System.Windows.Forms.DataGridView dgvStatistics;

        private System.Windows.Forms.RichTextBox rtbLog;

        protected override void Dispose(bool disposing)
        {
            if (disposing && (components != null))
                components.Dispose();

            base.Dispose(disposing);
        }

        private void InitializeComponent()
        {
            this.lblDay = new System.Windows.Forms.Label();
            this.btnNextDay = new System.Windows.Forms.Button();

            this.tabControl1 = new System.Windows.Forms.TabControl();
            this.tabRooms = new System.Windows.Forms.TabPage();
            this.tabOrders = new System.Windows.Forms.TabPage();
            this.tabStats = new System.Windows.Forms.TabPage();

            this.dgvRooms = new System.Windows.Forms.DataGridView();
            this.dgvOrders = new System.Windows.Forms.DataGridView();
            this.dgvStatistics = new System.Windows.Forms.DataGridView();

            this.rtbLog = new System.Windows.Forms.RichTextBox();

            this.tabControl1.SuspendLayout();
            this.tabRooms.SuspendLayout();
            this.tabOrders.SuspendLayout();
            this.tabStats.SuspendLayout();

            ((System.ComponentModel.ISupportInitialize)(this.dgvRooms)).BeginInit();
            ((System.ComponentModel.ISupportInitialize)(this.dgvOrders)).BeginInit();

```

```

((System.ComponentModel.ISupportInitialize)(this.dgvStatistics)).BeginInit()
;

this.SuspendLayout();

this.lblDay.AutoSize = true;
this.lblDay.Location = new System.Drawing.Point(12, 9);
this.lblDay.Name = "lblDay";
this.lblDay.Size = new System.Drawing.Size(100, 13);
this.lblDay.Anchor =
((System.Windows.Forms.AnchorStyles)(System.Windows.Forms.AnchorStyles.Top |
System.Windows.Forms.AnchorStyles.Left));
this.lblDay.Text = "Дата: ";

this.btnNextDay.Location = new System.Drawing.Point(120, 5);
this.btnNextDay.Name = "btnNextDay";
this.btnNextDay.Size = new System.Drawing.Size(130, 30);
this.btnNextDay.Anchor =
((System.Windows.Forms.AnchorStyles)(System.Windows.Forms.AnchorStyles.Top |
System.Windows.Forms.AnchorStyles.Left));
this.btnNextDay.Text = "Следующий день";
this.btnNextDay.Click += new
System.EventHandler(this.btnNextDay_Click);

this.tabControl1.Anchor =
((System.Windows.Forms.AnchorStyles)((((System.Windows.Forms.AnchorStyles.To
p | System.Windows.Forms.AnchorStyles.Bottom)
| System.Windows.Forms.AnchorStyles.Left)
| System.Windows.Forms.AnchorStyles.Right))));
this.tabControl1.Location = new System.Drawing.Point(12, 45);
this.tabControl1.Size = new System.Drawing.Size(960, 400);

this.tabRooms.Text = "Номера";
this.tabRooms.Controls.Add(this.dgvRooms);

this.tabOrders.Text = "Заказы";
this.tabOrders.Controls.Add(this.dgvOrders);

this.tabStats.Text = "Статистика";
this.tabStats.Controls.Add(this.dgvStatistics);

this.tabControl1.Controls.Add(this.tabRooms);
this.tabControl1.Controls.Add(this.tabOrders);
this.tabControl1.Controls.Add(this.tabStats);

this.dgvRooms.Dock = System.Windows.Forms.DockStyle.Fill;
this.dgvOrders.Dock = System.Windows.Forms.DockStyle.Fill;
this.dgvStatistics.Dock = System.Windows.Forms.DockStyle.Fill;

this.rtbLog.Anchor =
((System.Windows.Forms.AnchorStyles)((((System.Windows.Forms.AnchorStyles.Bot
tom | System.Windows.Forms.AnchorStyles.Left)
| System.Windows.Forms.AnchorStyles.Right))));
this.rtbLog.Location = new System.Drawing.Point(12, 460);
this.rtbLog.Size = new System.Drawing.Size(960, 180);
this.rtbLog.ReadOnly = true;

this.AutoScaleDimensions = new System.Drawing.SizeF(6F, 13F);
this.AutoScaleMode = System.Windows.Forms.AutoScaleMode.Font;
this.ClientSize = new System.Drawing.Size(984, 661);
this.MinimumSize = new System.Drawing.Size(800, 600);
this.Controls.Add(this.lblDay);
this.Controls.Add(this.btnNextDay);
this.Controls.Add(this.tabControl1);

```



```

        this.Controls.Add(this.rtbLog);

        this.Text = "Бронирование гостиницы";

        this.tabControl1.ResumeLayout(false);
        this.tabRooms.ResumeLayout(false);
        this.tabOrders.ResumeLayout(false);
        this.tabStats.ResumeLayout(false);

        ((System.ComponentModel.ISupportInitialize)(this.dgvRooms)).EndInit();
        ((System.ComponentModel.ISupportInitialize)(this.dgvOrders)).EndInit();
        ((System.ComponentModel.ISupportInitialize)(this.dgvStatistics)).EndInit();

        this.ResumeLayout(false);
        this.PerformLayout();
    }
}

```